Lab 1B: Regular Expressions

A *regular expression* is compact notation for a language (that is, a set of strings). Formally, a regular language is one of the following:

- The symbol $\varnothing$ (representing the empty set)
- Any string (representing the set containing only that string)
- $\boldsymbol{R} + \boldsymbol{S}$ for some regular expressions R and S (representing alternation / union)
- $\boldsymbol{R} \cdot \boldsymbol{S}$ or $\boldsymbol{RS}$ for some regular expressions R and S (representing concatenation)
- $\boldsymbol{R}^*$ for some regular expression R (representing Kleene closure / unbounded repetition)

In the absence of parentheses, Kleene closure has highest precedence, followed by concatenation. For example:

$$\texttt{0} + \texttt{RE}^* = \texttt{0} + \texttt{R}(\texttt{E}^*) = \{\texttt{0}, \texttt{R}, \texttt{RE}, \texttt{REE}, \texttt{REEE}, \texttt{REEEE}, \texttt{REEEEE}, \texttt{REEEEEE}, \texttt{REEEEEEE}, \ldots\}$$

$$(\texttt{0} + \texttt{RE})^* = \{\varepsilon, \texttt{0}, \texttt{RE}, \texttt{RERE}, \texttt{RERERE}, \texttt{RERERERE}, \texttt{RERERERERE}, \texttt{RERERERERERE}, \ldots\}$$

$$(\texttt{0} + \texttt{R})\texttt{E}^* = \{\texttt{0}, \texttt{R}, \texttt{0E}, \texttt{RE}, \texttt{0EE}, \texttt{REE}, \texttt{0EEE}, \texttt{REEE}, \texttt{0EEEE}, \texttt{REEEE}, \texttt{0EEEEE}, \ldots\}$$

$$(\texttt{0} + \texttt{RE})^* = \{\varepsilon, \texttt{0}, \texttt{00}, \texttt{RE}, \texttt{000}, \texttt{0RE}, \texttt{RE0}, \texttt{0000}, \texttt{00RE}, \texttt{0RE0}, \texttt{RE00}, \texttt{RERE}, \ldots\}$$

Give regular expressions for each of the following languages over the binary alphabet {0, 1}. (For extra practice, find multiple regular expressions for each language.)

0. All strings.

$$\boxed{(\texttt{0} + \texttt{1})^*}$$

1. All strings containing the substring 000.

$$\boxed{\texttt{((0 + 1)* 000 (0 + 1)*)}}$$

2. All strings *not* containing the substring 000.

$$\boxed{(\texttt{1} + \texttt{01} + \texttt{001})^*(\varepsilon + \texttt{0} + \texttt{00})}$$

3. All strings in which every run of 0s has length at least 3.
4. All strings in which the *last* 1 appears before the *first* substring 000.
5. All strings containing at least three 0s.
6. Every string except 000. *[Hint: Don't try to be clever.]*